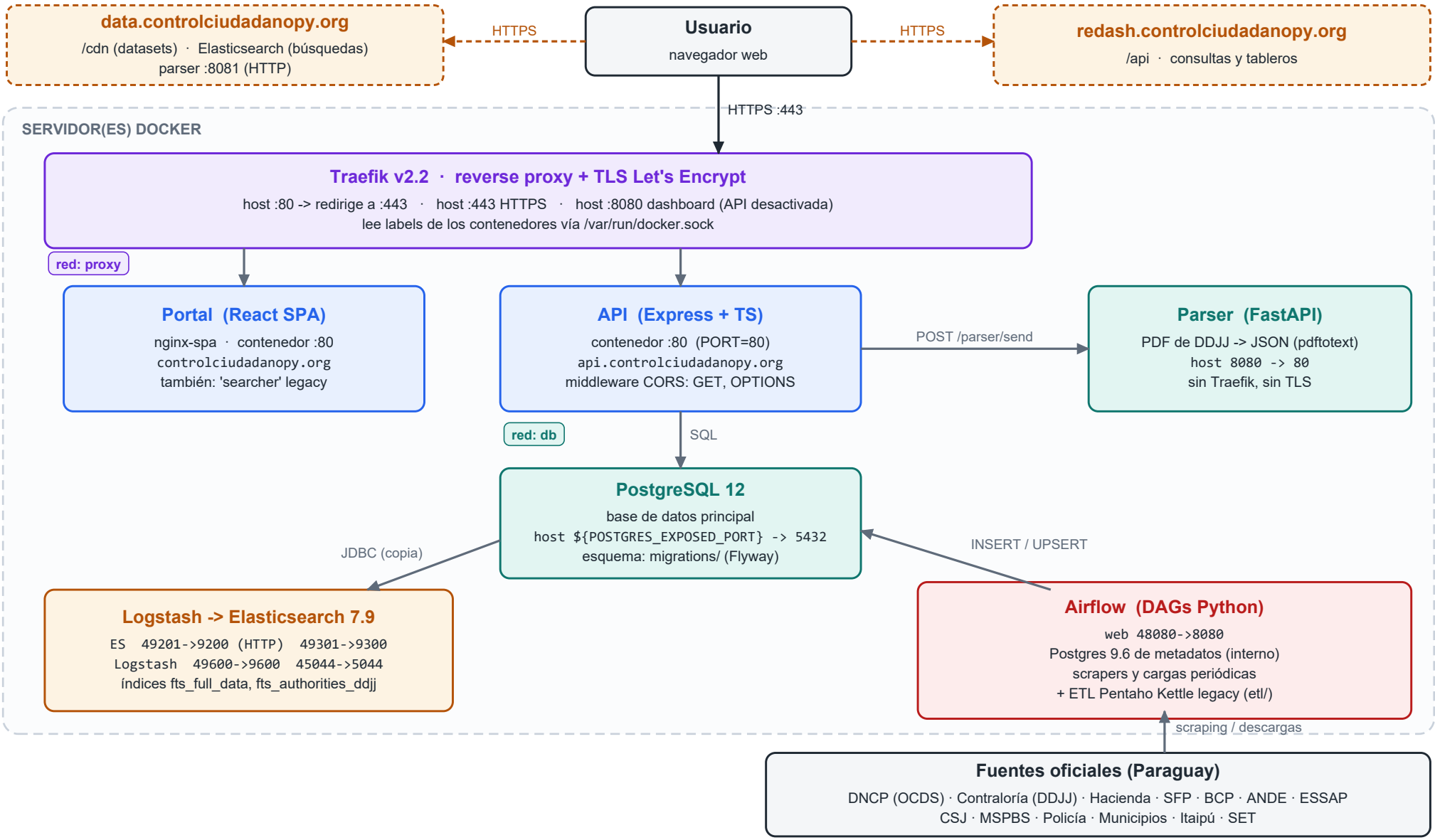
Control Ciudadano — Arquitectura

Portal de datos abiertos para el control de recursos públicos de Paraguay (InstIDEA). Documento generado el 10/11/2026 a partir del repositorio (último commit: 12/07/2023).



Líneas punteadas: servicios a los que el navegador llama directo, fuera del Traefik descrito en el repo. Formato de puertos: **host -> contenedor**.

1. Componentes del repositorio

Carpeta	Qué es	Tecnología	Cómo se ejecuta
portal/	Sitio web (SPA): compras COVID/OCDS, declaraciones juradas, autoridades, análisis de patrimonio	React 16, TypeScript, Ant Design, Nivo, ReactiveSearch	Imagen nginx-spa con el build estático
api/	API REST: /api/ocds/*, /api/contralory/declarations, /api/people/:doc/declarations, /api/analysis/*	Node 14, Express, TypeScript, pg, Mocha	Contenedor detrás de Traefik
scripts/python/parser/	Microservicio que convierte PDF de declaraciones juradas en datos	FastAPI, pdftotext, SQLAlchemy	Imagen microservice_parser
scripts/python/airflow/	DAGs: scrapers de Contraloría, CSJ, MSPBS, Policía, municipios, SFP, Itaipú…	Apache Airflow (puckel)	Imagen dags
scripts/python/similarity/	Similitud entre proveedores (mismos datos de contacto)	Python	Manual
etl/	ETL legacy: ANDE, ESSAP, BCP, Hacienda, inflación, SET, Pytyvõ, Ñangareko	Pentaho Kettle (.ktr/.kjb)	Manual
migrations/	Esquema de la base (V1–V10 + scripts R_)	SQL / Flyway	Manual
infra/	docker-compose: base, API, Traefik, parser, ELK, Airflow	Docker Compose	En el servidor

2. Recorrido de una visita

1. El navegador pide *controlciudadanopy.org*. Traefik termina el HTTPS y entrega la SPA desde el contenedor del portal.
2. La SPA llama a *api.controlciudadanopy.org*. Traefik aplica CORS y reenvía a la API Express, que consulta PostgreSQL por la red **db**.
3. Las búsquedas de texto (ReactiveSearch) van a Elasticsearch en *data.controlciudadanopy.org*; Logstash lo sincroniza desde Postgres por JDBC.
4. Para declaraciones juradas, la API envía el PDF al parser (`http://data.controlciudadanopy.org:8081/parser/send`) y guarda el resultado.
5. Airflow corre periódicamente los scrapers de las fuentes oficiales y escribe en PostgreSQL.

3. Puertos y dominios

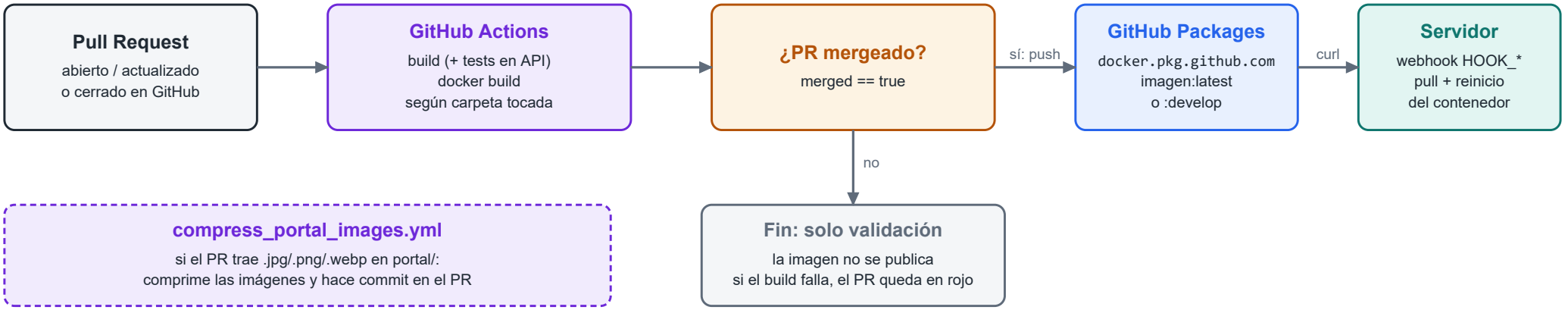
Servicio	Archivo	Puerto host	Puerto contenedor	Acceso público	Notas
Traefik	infra/docker-compose.yml	80, 443, 8080	80, 443, 8080	Sí (punto de entrada)	80 redirige a 443. 8080 es el dashboard, con --api.insecure comentado.
Portal (nginx-spa)	no está en el repo	—	80	controlciudadanopy.org (vía Traefik)	El compose real vive en el servidor.
searcher (legacy)	infra/docker-compose.yml	—	80	controlciudadanopy.org, www (vía Traefik)	Versión vieja del front servida con 'npx serve'.
API Express	infra/docker-compose.yml	—	80 (PORT=80)	api.controlciudadanopy.org (vía Traefik)	En local el puerto por defecto es 3000.
PostgreSQL 12	infra/docker-compose.yml	POSTGRES_ EXPOSED_PORT (.env)	5432	Depende del .env	Red db. max_connections=500.
Parser FastAPI	infra/services/docker-compose.yml	8080	80	Directo, sin TLS	La API lo llama en :8081: el despliegue real difiere del compose.
Elasticsearch 7.9	infra/elk/docker-compose.yml	49201, 49301	9200, 9300	Directo; el portal usa data.controlciudadanopy.org	9200 = API HTTP, 9300 = transporte entre nodos.
Logstash	infra/elk/docker-compose.yml	49600, 45044	9600, 5044	Directo	9600 = API de monitoreo, 5044 = entrada Beats.
Airflow webserver	infra/airflow/docker-compose.yml	48080	8080	Directo, con login por contraseña	
Postgres 9.6 (Airflow)	infra/airflow/docker-compose.yml	—	5432	No	Solo metadatos de Airflow.
Portal en desarrollo	portal/package.json	3000	—	Local	yarn start. CORS de la API permite localhost:3000.

Dominios y redes

- **controlciudadanopy.org / www**: portal. **api.**: API Express. **data.**: CDN de datasets, Elasticsearch y parser. **redash.**: Redash.
- **Red proxy** (externa): une Traefik con los servicios públicos. **Red db** (externa): une la API con PostgreSQL. Hay que crearlas antes: `docker network create proxy` y `docker network create db`.
- Traefik y el parser usan los dos el puerto host 8080: si estuvieran en el mismo servidor chocarían, así que probablemente están en máquinas distintas.

4. Qué pasa cuando se sube algo a GitHub (CI/CD)

Los workflows se disparan **solo con pull requests** que tocan su carpeta. Un push directo a master no despliega nada.



Workflow	Se dispara si el PR toca	En cada PR	Al mergear publica	Webhook de despliegue
portal.yml	portal/**	yarn, npm run build, docker build	imagen portal	HOOK_PORTAL / HOOK_PORTAL_DEVELOP
api.yml	api/**	npm ci, tsc, npm test, docker build	imagen api	HOOK_API / HOOK_API_DEVELOP
airflow.yml	scripts/python/airflow/**	docker build	imagen dags	HOOK_DAGS
microservice_parser.yml	scripts/python/parser/**	docker build	imagen microservice_parser	HOOK_SERVICE_DJBR
compress_portal_images.yml	imágenes .jpg/.png/.webp en portal/	comprime y hace commit en el PR	—	—
(ninguno)	etl/, migrations/, infra/, similarity/, dncp/	nada	—	Se aplica a mano en el servidor

Tags: master -> **latest**, develop -> **develop**. Ojo: en un pull_request, github.ref es `refs/pull/N/merge`, así que el tag calculado sería 'merge'. Conviene revisar los logs de Actions para confirmar que los webhooks realmente se llaman.

5. Riesgos y pendientes

Tema	Detalle	Sugerencia
Versiones sin soporte	Node 14, TypeScript 3.9, React 16, Traefik 2.2, Elasticsearch 7.9, Postgres 9.6 y 12, FastAPI 0.63	Actualizar por capas: primero CI e imágenes base, después las dependencias.
CI obsoleto	actions/checkout@v2, setup-node@v1 y registro docker.pkg.github.com (reemplazado por ghcr.io)	Migrar a ghcr.io y a las versiones actuales de las actions.
Puertos expuestos	Elasticsearch, Logstash, Airflow y parser publican puertos en el host sin TLS	Cerrarlos con firewall o ponerlos detrás de Traefik con autenticación.
Docker socket	Traefik monta /var/run/docker.sock con escritura	Montarlo :ro o usar un docker-socket-proxy.
URLs fijas en el código	portal/src/SimpleApi.ts y RedashAPI.ts; parser :8081 en DJBRParserApi.ts	Pasarlas a variables REACT_APP_* y de entorno.
Config fuera del repo	.env, compose del portal, webhooks y proxy de data.controlciudadanopy.org	Documentarlos o versionarlos (sin secretos).
Duplicaciones	ETL Kettle y Airflow conviven; similarity/ duplicado dentro de los DAGs	Definir una sola fuente de verdad.

6. Comandos útiles

- Portal en local: `cd portal && yarn install && yarn start` (http://localhost:3000)
- Build y contenedor del portal: `yarn build && docker build -t portal . && docker run -p 8080:80 portal`
- API en local: `cd api && npm ci && npm start` · tests: `npm test`
- Infra: `docker network create proxy; docker network create db; docker compose -f infra/docker-compose.yml up -d`